

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2025 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Model 2025	2
SUBIECTUL I (30 de puncte)	2
1. Structura de date: Arbori	2
2. Ergonomia postului de lucru	3
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Lanț muntos (Creastă)	3
2. Algoritm eficient: Tije discuri	5
SUBIECTUL al III-lea (30 de puncte)	7
1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)	7
2. Evaluare: Itemi obiectivi (Alegere multiplă)	7
II. Rezolvare Titularizare Varianta 3 (15 iulie 2025)	8
SUBIECTUL I (30 de puncte)	8
1. Structura de date: Coadă	8
a) Accesul la elemente	8
b) Descriere și Exemplificare pentru o coadă cu 4 elemente	8
1. Reprezentarea Secvențială (Vector fix și indicii front/rear)	8
2. Reprezentarea Înlănțuită (Noduri dinamice)	8
c) Utilizare într-o problemă: Algoritm Lee	8
2. Arhitectura sistemelor de calcul: Unitatea Centrală de Prelucrare (UCP)	10
SUBIECTUL al II-lea (30 de puncte)	10
1. Programare Backtracking: Combinări bancnote	10
2. Algoritmi eficienți: Suprapunere Programări Cabinet Medic	12
SUBIECTUL al III-lea (30 de puncte)	13
1. Proiectare Test de Evaluare și Barem (Secvența A)	13
2. Proiectarea unei strategii didactice pentru Secvența B (Animații și Modele 3D)	13

I. Rezolvare Titularizare Model 2025

SUBIECTUL I (30 de puncte)

1. Structura de date: Arbori

- **Graf neorientat:** Pereche ordonată $G = (V, E)$, unde V este mulțimea nodurilor și E mulțimea muchiilor.
- **Lanț:** Succesiune de noduri interconectate prin muchii distincte.
- **Ciclu:** Un lanț simplu în care nodul de pornire coincide cu cel de sosire.
- **Proprietăți arbore:**
 1. Conex și fără cicluri.
 2. Fără cicluri și are $N - 1$ muchii.
 3. Conex și are $N - 1$ muchii.
 4. Între oricare două noduri există un lanț simplu unic.
 5. Dacă se elimină o muchie, devine neconex; dacă se adaugă o muchie, se formează exact un ciclu.

• Problemă (Parcurgerea în lățime a unui arbore):

► C++:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
vector<int> adj[100];
bool viz[100];
void BFS(int start) {
    queue<int> Q;
    Q.push(start);
    viz[start] = true;
    while(!Q.empty()) {
        int u = Q.front(); Q.pop();
        cout << u << " ";
        for(int v : adj[u])
            if(!viz[v]) { viz[v] = true; Q.push(v); }
    }
}
```

► Pascal:

```
program BFSArbore;
const NMAX = 100;
var
    n, i, x, y, start, st, dr, u, v: integer;
    a: array[1..NMAX, 1..NMAX] of integer;
    grad, q: array[1..NMAX] of integer;
    viz: array[1..NMAX] of boolean;
begin
    read(n);
    for i := 1 to n - 1 do
        begin
            read(x, y);
            grad[x] := grad[x] + 1; a[x, grad[x]] := y;
            grad[y] := grad[y] + 1; a[y, grad[y]] := x;
```

```

end;
read(start);
st := 1; dr := 1; q[1] := start; viz[start] := true;
while st <= dr do
begin
    u := q[st]; st := st + 1;
    write(u, ' ');
    for i := 1 to grad[u] do
    begin
        v := a[u, i];
        if not viz[v] then
        begin
            dr := dr + 1; q[dr] := v; viz[v] := true;
        end;
    end;
end;
end.

```

2. Ergonomia postului de lucru

- **Noțiuni preliminare:** Sistemul de calcul este ansamblul hardware-software care prelucrează automat date. Dispozitivele periferice sunt echipamente de intrare, ieșire sau intrare/ieșire care asigură comunicarea dintre utilizator și calculator.
- **Dispozitive periferice cu impact asupra sănătății:**
 1. **Monitorul:** are funcția de afișare a informațiilor. Poziționarea greșită, reflexiile, luminozitatea nepotrivită sau distanța prea mică pot cauza oboseală oculară, dureri de cap și postură incorectă.
 2. **Tastatura și mouse-ul:** permit introducerea datelor și controlul interfeței. Utilizarea lor îndelungată, cu încheieturile tensionate, poate produce dureri musculare, suprasolicitare și sindrom de tunel carpian.
- **Măsuri ergonomice:**
 1. Monitorul se poziționează la 50-70 cm de ochi, cu partea superioară aproximativ la nivelul privirii.
 2. Scaunul și masa se reglează astfel încât spatele să fie sprijinit, iar coatele și genunchii să formeze un unghi apropiat de 90 de grade.
 3. Se fac pauze scurte și regulate; privirea se mută periodic de la ecran, iar mâinile și umerii se relaxează.

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Lanț muntos (Creastă)

Soluție C++:

```

#include <iostream>
#include <cmath>
using namespace std;

int varf(int n, int a[50][50], int k) {
    int r = k - 1;
    int max_val = a[r][0];
    int max_pos = 0;
    for (int j = 1; j < n; ++j) {
        if (a[r][j] > max_val) {
            max_val = a[r][j];

```

```

        max_pos = j;
    }
}
for (int j = 0; j < n; ++j) {
    if (j != max_pos && a[r][j] == max_val) return 0;
}
for (int j = 0; j < max_pos; ++j) {
    if (a[r][j] >= a[r][j+1]) return 0;
}
for (int j = max_pos; j < n - 1; ++j) {
    if (a[r][j] <= a[r][j+1]) return 0;
}
return max_pos + 1;
}

int main() {
    int n, a[50][50];
    if (!(cin >> n)) return 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> a[i][j];

    bool e_creasta = true;
    int prev_p = varf(n, a, 1);
    if (prev_p == 0) e_creasta = false;

    for (int k = 2; k <= n && e_creasta; ++k) {
        int curr_p = varf(n, a, k);
        if (curr_p == 0 || abs(curr_p - prev_p) > 1) {
            e_creasta = false;
        }
        prev_p = curr_p;
    }

    if (e_creasta) cout << "DA" << endl;
    else cout << "NU" << endl;

    return 0;
}

```

Soluție Pascal:

```

program LantMuntos;
type
    TMatrice = array[1..50, 1..50] of integer;
var
    n, i, j, prev_p, curr_p: integer;
    a: TMatrice;
    e_creasta: boolean;

function varf(n: integer; var a: TMatrice; k: integer): integer;
var
    max_val, max_pos, col: integer;
    is_mountain: boolean;
begin
    max_val := a[k, 1];
    max_pos := 1;

```

```

for col := 2 to n do
    if a[k, col] > max_val then
        begin
            max_val := a[k, col];
            max_pos := col;
        end;

is_mountain := true;
for col := 1 to n do
    if (col <> max_pos) and (a[k, col] = max_val) then
        is_mountain := false;

for col := 1 to max_pos - 1 do
    if a[k, col] >= a[k, col + 1] then
        is_mountain := false;

for col := max_pos to n - 1 do
    if a[k, col] <= a[k, col + 1] then
        is_mountain := false;

if is_mountain then varf := max_pos
else varf := 0;
end;

begin
    readln(n);
    for i := 1 to n do
        for j := 1 to n do
            read(a[i, j]);

e_creasta := true;
prev_p := varf(n, a, 1);
if prev_p = 0 then e_creasta := false;

i := 2;
while (i <= n) and e_creasta do
    begin
        curr_p := varf(n, a, i);
        if (curr_p = 0) or (abs(curr_p - prev_p) > 1) then
            e_creasta := false;
        prev_p := curr_p;
        i := i + 1;
    end;

if e_creasta then writeln('DA')
else writeln('NU');
end.

```

2. Algoritm eficient: Tije discuri

- **Algoritmul:** Rămâne o listă a diametrelor din vârful fiecărei tije ocupate. Această listă este întotdeauna sortată crescător. Căutăm binar prima tijă adecvată. Complexitate: $O(N \log N)$ timp și $O(N)$ spațiu.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu2025.txt");
    if (!fin) return 1;
    vector<int> R;
    int d;
    while (fin >> d) {
        auto it = lower_bound(R.begin(), R.end(), d);
        if (it == R.end()) R.push_back(d);
        else *it = d;
    }
    fin.close();
    cout << R.size() << endl;
    return 0;
}
```

Soluție Pascal:

```
program TijeDiscuri;
var
    fin: text;
    R: array[1..100000] of integer;
    num_rods, d, pos, st, dr, mid: integer;
begin
    assign(fin, 'titu2025.txt'); reset(fin);
    num_rods := 0;
    while not eof(fin) do
        begin
            read(fin, d);
            st := 1; dr := num_rods; pos := -1;
            while st <= dr do
                begin
                    mid := (st + dr) div 2;
                    if R[mid] >= d then
                        begin
                            pos := mid; dr := mid - 1;
                        end
                    else st := mid + 1;
                end;
            if pos = -1 then
                begin
                    num_rods := num_rods + 1;
                    R[num_rods] := d;
                end
            else R[pos] := d;
            end;
        close(fin);
        writeln(num_rods);
    end.
```

SUBIECTUL al III-lea (30 de puncte)

1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)

- **Tip lecție:** Lecție de formare a priceperilor și deprinderilor practice.
- **Caracteristici:** accent pe exersare directă în aplicație; feedback imediat prin observarea modificărilor pe ecran; sarcini gradate de la operații simple la produs final.
- **Mijloc de învățământ:** calculator cu procesor de text și videoproiector.
- **Metodă:** exercițiul practic dirijat.
- **Formă de organizare:** activitate individuală la calculator, cu momente frontale de demonstrație.
- **Activitate:** formatarea unui text brut pentru a obține un afiș/anunț lizibil.
- **Scenariu:** Profesorul proiectează un text neformatat și demonstrează selectarea textului, schimbarea fontului, aplicarea stilurilor bold/italic/subliniat și alinierea. Elevii primesc același text, aplică cerințe graduale și compară variantele obținute. Profesorul corectează folosirea excesivă a formătărilor și fixează ideea că formatarea trebuie să crească lizibilitatea.

2. Evaluare: Itemi obiectivi (Alegere multiplă)

- **Avantaje:** corectare rapidă și obiectivă; acoperirea unui volum mare de conținut; posibilitatea utilizării în teste standardizate.
- **Limită:** itemii obiectivi evaluează mai greu argumentarea și construcția completă a unei soluții.
- **Reguli de proiectare:** enunț clar, un singur răspuns corect, distractori plauzibili și omogeni.
- **Item pentru Backtracking (Secvența A):** Care este structura spațiului stărilor pentru generarea permutărilor? A. Produs cartezian. | B. Vector de lungime constantă. | C. Arbore de căutare. | D. Graf conex. Răspuns: C.
- **Item pentru formatarea textului (Secvența B):** Afirmația „Comanda Bold modifică dimensiunea caracterelor selectate” este adevărată sau falsă? Răspuns: Fals. Bold modifică grosimea caracterelor, nu dimensiunea lor.

II. Rezolvare Titularizare Varianta 3 (15 iulie 2025)

SUBIECTUL I (30 de puncte)

1. Structura de date: Coadă

a) Accesul la elemente

O coadă este o structură de date de tip **FIFO (First-In, First-Out)**. Accesul direct este permis doar la extremitățile cozii:

- **Față (Front):** de unde se citesc și se elimină elementele.
- **Spate (Rear):** unde se adaugă elementele noi.

b) Descriere și Exemplificare pentru o coadă cu 4 elemente

Fie coada în care se inserează succesiv valorile A, B, C, D, apoi se realizează o ștergere.

1. Reprezentarea Secvențială (Vector fix și indicii front/rear)

• Cod C++:

```
char Q[100];
int front = 0, rear = 0;
Q[rear++] = 'A'; // Inserare
char sters = Q[front++]; // Stergere
```

• Cod Pascal:

```
var
  Q: array[0..99] of char;
  front: integer = 0;
  rear: integer = 0;
begin
  Q[rear] := 'A'; rear := rear + 1; // Inserare
  sters := Q[front]; front := front + 1; // Stergere
end;
```

2. Reprezentarea Înlănțuită (Noduri dinamice)

• Cod C++:

```
struct Nod { char info; Nod* urm; };
Nod *prim = nullptr, *ultim = nullptr;
```

• Cod Pascal:

```
type
  PNod = ^TNod;
  TNod = record info: char; urm: PNod; end;
var
  prim: PNod = nil;
  ultim: PNod = nil;
```

c) Utilizare într-o problemă: Algoritmul Lee

- **Enunț:** Determinarea numărului minim de pași de la o poziție de start la o destinație într-un labirint binar.

• Cod C++:

```
#include <iostream>
using namespace std;
```



```

struct Coordonate {
    int x, y;
};

Coordonate Q[10000];
int frontIdx = 0, rearIdx = 0;

int labirint[100][100], dist[100][100];
int n = 5, m = 5;
int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

int lee(int xs, int ys, int xf, int yf) {
    dist[xs][ys] = 1;
    Q[rearIdx++] = {xs, ys};

    while (frontIdx < rearIdx) {
        Coordonate c = Q[frontIdx++];
        if (c.x == xf && c.y == yf) return dist[xf][yf] - 1;

        for (int i = 0; i < 4; ++i) {
            int nx = c.x + dx[i];
            int ny = c.y + dy[i];
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m) {
                if (labirint[nx][ny] == 0 && dist[nx][ny] == 0) {
                    dist[nx][ny] = dist[c.x][c.y] + 1;
                    Q[rearIdx++] = {nx, ny};
                }
            }
        }
    }
    return -1;
}

int main() {
    int rez = lee(1, 1, 3, 3);
    cout << "Pasi: " << rez << endl;
    return 0;
}

```

• **Cod Pascal:**

```

program LeeCoadă;
type
    Coordonate = record
        x, y: integer;
    end;
var
    Q: array[0..9999] of Coordonate;
    frontIdx: integer = 0;
    rearIdx: integer = 0;
    labirint: array[1..100, 1..100] of integer;
    dist: array[1..100, 1..100] of integer;
    n: integer = 5;
    m: integer = 5;
    dx: array[0..3] of integer = (-1, 0, 1, 0);
    dy: array[0..3] of integer = (0, 1, 0, -1);

```

```

function lee(xs, ys, xf, yf: integer): integer;
var
  c: Coordonate;
  i, nx, ny: integer;
begin
  dist[xs, ys] := 1;
  Q[rearIdx].x := xs; Q[rearIdx].y := ys;
  rearIdx := rearIdx + 1;

  while frontIdx < rearIdx do
  begin
    c := Q[frontIdx];
    frontIdx := frontIdx + 1;
    if (c.x = xf) and (c.y = yf) then
    begin
      lee := dist[xf, yf] - 1;
      exit;
    end;
    for i := 0 to 3 do
    begin
      nx := c.x + dx[i];
      ny := c.y + dy[i];
      if (nx >= 1) and (nx <= n) and (ny >= 1) and (ny <= m) then
      begin
        if (labirint[nx, ny] = 0) and (dist[nx, ny] = 0) then
        begin
          dist[nx, ny] := dist[c.x, c.y] + 1;
          Q[rearIdx].x := nx; Q[rearIdx].y := ny;
          rearIdx := rearIdx + 1;
        end;
      end;
    end;
  end;
  lee := -1;
end;

begin
  writeln('Pasi: ', lee(1, 1, 3, 3));
end.

```

2. Arhitectura sistemelor de calcul: Unitatea Centrală de Prelucrare (UCP)

- **Magistrale:** Date (translație date), Adrese (locație RAM), Control (semnale R/W și tact).
- **Componente:** ALU (aritmetică și logică), UCC (comandă și control).
- **Parametri:** Frecvență tact 5.2 GHz, Număr nuclee 16, Cache L3 32 MB.

SUBIECTUL al II-lea (30 de puncte)

1. Programare Backtracking: Combinări bancnote

Păstrăm cupiurile în ordinea în care au fost citite în tabloul c, deoarece subprogramul tipar trebuie să afișeze bancnotele utilizate în această ordine.

Soluție C++:

```
#include <iostream>
using namespace std;

int n;
int c[10], b[10];

void tipar() {
    for (int i = 0; i < n; ++i) {
        if (b[i] > 0) {
            cout << b[i] << "x" << c[i] << (c[i] == 1 ? "leu " : "lei ");
        }
    }
    cout << "\n";
}

void back(int idx, int sum) {
    if (sum == 0) { tipar(); return; }
    if (idx < 0) return;
    int max_b = sum / c[idx];
    for (int bills = max_b; bills >= 0; --bills) {
        b[idx] = bills;
        back(idx - 1, sum - bills * c[idx]);
    }
    b[idx] = 0;
}

int main() {
    int s;
    cin >> s >> n;
    for (int i = 0; i < n; ++i) { cin >> c[i]; b[i] = 0; }
    back(n - 1, s);
    return 0;
}
```

Soluție Pascal:

```
program CombinariBancnote;
var
    s, i: integer;
    n: integer;
    c, b: array[0..9] of integer;

procedure tipar;
var k: integer;
begin
    for k := 0 to n - 1 do
        begin
            if b[k] > 0 then
                begin
                    if c[k] = 1 then write(b[k], 'x', c[k], 'leu ')
                    else write(b[k], 'x', c[k], 'lei ');
                end;
            end;
        writeln;
    end;
```

```

procedure back(idx, sum: integer);
var bills, max_b: integer;
begin
  if sum = 0 then begin tipar; exit; end;
  if idx < 0 then exit;
  max_b := sum div c[idx];
  for bills := max_b downto 0 do
  begin
    b[idx] := bills;
    back(idx - 1, sum - bills * c[idx]);
  end;
  b[idx] := 0;
end;

begin
  read(s, n);
  for i := 0 to n - 1 do begin read(c[i]); b[i] := 0; end;
  back(n - 1, s);
end.

```

2. Algoritmi eficienți: Suprapunere Programări Cabinet Medic

Metoda: Programările sunt parcurse în ordinea din fișier. Reținem ora programării curente păstrate. Dacă următoarea programare are aceeași oră, ea se suprapune și trebuie actualizată, deci creștem contorul. Dacă ora este diferită, actualizăm ora curentă păstrată. Algoritmul presupune ordinea cronologică a programărilor din fișier; pentru date nesortate se poate folosi un tabel de frecvențe pe minutele zilei.

Eficiență: Parcurgem fișierul o singură dată, deci timpul este $O(n)$, iar memoria suplimentară este $O(1)$.

Soluție C++:

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
  ifstream fin("titu2025.txt");
  int current, val, rescheduled = 0;
  if (!(fin >> current)) return 0;
  while (fin >> val) {
    if (val == current) rescheduled++;
    else current = val;
  }
  cout << rescheduled << endl;
  return 0;
}

```

Soluție Pascal:

```

program EvaluareSuprapuneri;
var
  fin: text;
  current, val, rescheduled: longint;
begin
  assign(fin, 'titu2025.txt'); reset(fin);

```

```

if eof(fin) then begin writeln(0); close(fin); halt; end;
read(fin, current);
rescheduled := 0;
while not eof(fin) do
begin
    read(fin, val);
    if val = current then rescheduled := rescheduled + 1
    else current := val;
end;
close(fin);
writeln(rescheduled);
end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Proiectare Test de Evaluare și Barem (Secvența A)

Item	Tip & Cerință (C++ / Pascal)
Item 1	Alegere Multiplă. Parametri transmiși prin valoare vs referință. Răspuns: B.
Item 2	Completare spații libere: determinare dacă un număr este par.
Item 3	Răspuns Scurt: Avantajele modularizării codului.
Item 4	Întrebare structurată pentru urmărirea recursivității.
Item 5	Subiectiv: Calculul sumei cifrelor impare ale unui număr natural.

2. Proiectarea unei strategii didactice pentru Secvența B (Animații și Modele 3D)

- **Mijloc:** Tinkercad.
- **Metoda:** Învățarea prin descoperire.
- **Activitate:** Crearea și modelarea unei animații de translație 3D.